NutriScan: Aplicação Móvel Android para Registo e Acompanhamento Nutricional

Diogo Reis
Número de estudante: 1250895

Desenvolvimento de Software e Sistemas Móveis (DSSMV) · Ano lectivo 2026/2027

Resumo

O NutriScan é uma aplicação móvel nativa para Android destinada ao registo de alimentos e refeições e ao acompanhamento de valores nutricionais. A informação dos produtos pode ser obtida através da leitura de códigos de barras, com consulta à API Open Food Facts, ou introduzida manualmente. A aplicação calcula calorias, proteínas, hidratos de carbono e gorduras por porção, agrega os valores das refeições e compara os totais diários com metas definidas pelo utilizador. Inclui ainda alimentos e refeições guardados, calendário, histórico diário, perfil e conquistas. A autenticação é assegurada pelo Firebase Authentication e os dados são persistidos por utilizador na Firebase Realtime Database.

Palavras-chave: Android · Nutrição · Java · Open Food Facts · Firebase · Aplicação móvel

1 Introdução

A análise funcional e técnica apresentada neste artigo baseia-se no código do repositório NutriScan [2].

O NutriScan foi desenvolvido no âmbito da unidade curricular Desenvolvimento de Software e Sistemas Móveis (DSSMV), da Licenciatura em Engenharia de Telecomunicações e Informática do ISEP. O projecto consiste numa aplicação Android nativa que permite registar alimentos e refeições e acompanhar a informação nutricional ao longo do tempo. A implementação utiliza Java e comunica com serviços externos, cumprindo a orientação do enunciado para uma aplicação móvel que interage com o mundo exterior [1].

1.1 Contexto e Objectivos

A aplicação procura simplificar o registo alimentar. Para cada alimento, guarda valores nutricionais por 100 g; quando o utilizador indica a quantidade consumida, calcula os valores correspondentes à porção. Os totais são agregados por refeição e por dia, podendo ser comparados com metas individuais.

Os principais objectivos são: (i) consultar produtos por código de barras; (ii) permitir a introdução manual de alimentos; (iii) registar, consultar, apagar e reutilizar refeições; (iv) definir metas mínimas ou máximas para nutrientes; e (v) apresentar progresso diário, calendário, histórico e conquistas.

1.2 Tecnologias

Tabela 1. Tecnologias e bibliotecas identificadas na configuração do projecto.

Componente	Tecnologia observada
Plataforma e linguagem	Android nativo · Java 11 · AndroidX
Interface	Activities Android · layouts XML · Material Components
Câmara e códigos de barras	CameraX 1.4.2 · ML Kit Barcode Scanning 17.3.0
Autenticação e persistência	Firebase Authentication · Firebase Realtime Database (REST)
Consulta de produtos	Open Food Facts API v2
Testes	JUnit 4.13.2 · AndroidX Test · Espresso

2 Análise

2.1 Actores e Sistemas Externos

O actor principal é o Utilizador, que interage com a aplicação para gerir a sua informação alimentar. A Câmara Android fornece as imagens necessárias à leitura de códigos. A API Open Food Facts fornece informação de produtos. O Firebase Authentication gere a sessão e a Firebase Realtime Database guarda os dados associados ao utilizador autenticado.

2.2 Requisitos Funcionais

Tabela 2. Requisitos funcionais identificados na implementação.

ID	Requisito
RF01	Criar conta, iniciar e terminar sessão com email e palavra-passe.
RF02	Ler códigos EAN/UPC e consultar dados do produto online.
RF03	Registar alimentos manualmente com os respectivos nutrientes.
RF04	Definir porção em gramas e calcular os nutrientes.
RF05	Registar, consultar e apagar refeições por data.
RF06	Guardar e reutilizar alimentos e modelos de refeição.
RF07	Definir metas mínimas/máximas de calorias, proteínas, hidratos e gorduras.
RF08	Consultar progresso diário, calendário e estatísticas mensais.
RF09	Gerir fotografia de perfil e consultar conquistas.

2.3 Requisitos Não Funcionais

Tabela 3. Requisitos não funcionais observáveis no código.

ID	Requisito
RNF01	Compatibilidade declarada com Android API 24 ou superior (minSdk 24).
RNF02	Os pedidos à base de dados utilizam a sessão Firebase e o respectivo ID token.
RNF03	As operações de rede são executadas em segundo plano e os resultados são encaminhados para a thread principal.
RNF04	A aplicação trata situações como falhas de rede, produto não encontrado e recusa da permissão da câmara.
RNF05	Os valores nutricionais podem ser introduzidos por controlos deslizantes e campos numéricos.

2.4 Casos de Utilização

Os principais casos de utilização são: UC01 - gerir autenticação; UC02 - adicionar alimento por código de barras; UC03 - adicionar alimento manualmente; UC04 - definir quantidade e adicionar alimento à refeição; UC05 - concluir e registar refeição; UC06 - guardar refeição como modelo; UC07 - reutilizar refeição guardada; UC08 - consultar e apagar alimentos guardados; UC09 - definir metas nutricionais; UC10 - consultar painel e calendário; UC11 - consultar detalhe diário e apagar refeição; UC12 - consultar conquistas; e UC13 - gerir fotografia de perfil.

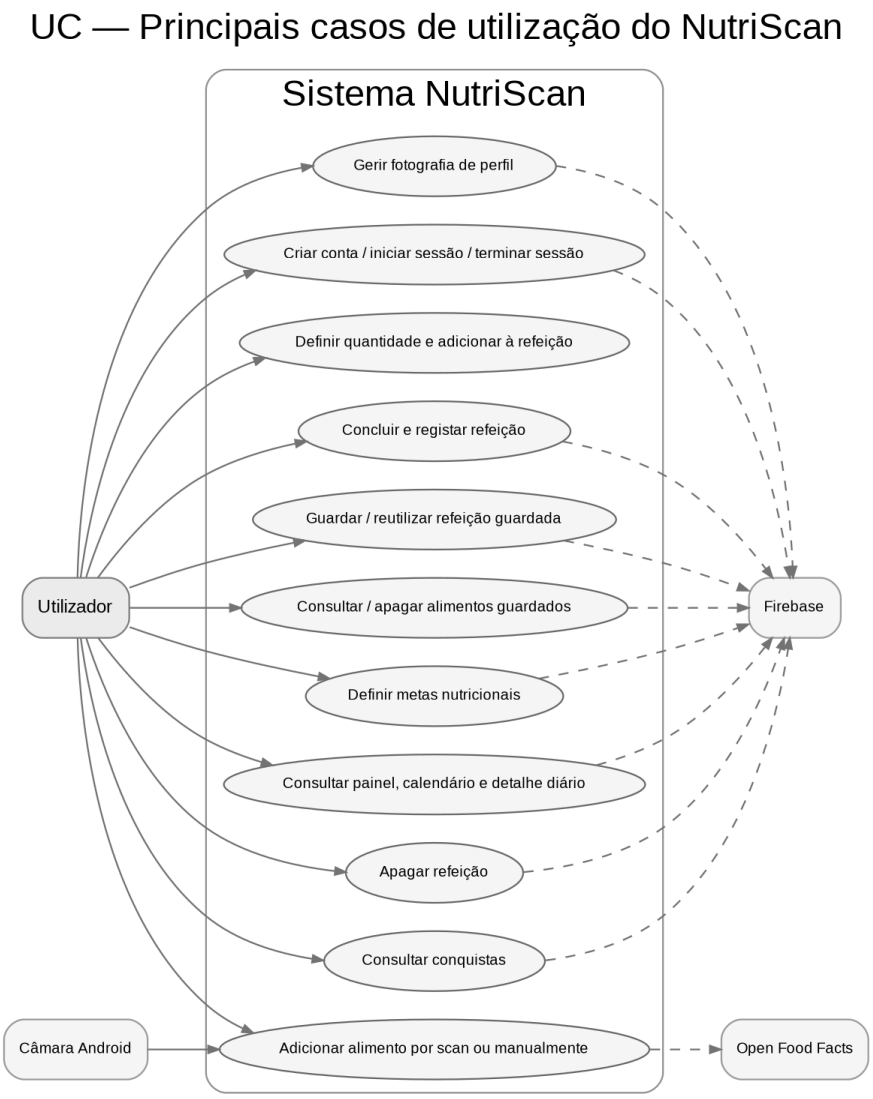


Fig. 1. Diagrama dos principais casos de utilização do NutriScan.

3 Design

3.1 Organização Modular

A aplicação organiza-se em Activities para a interface, serviços para as operações da aplicação, modelos para representar os dados e classes auxiliares para cálculos, datas, imagens, serialização e apresentação. As Activities invocam directamente serviços e

recebem resultados através de callbacks. Não foi identificada uma implementação formal de MVVM.

A camada de serviços inclui FoodService, MealService, SavedMealService, GoalService, ProfileService, AchievementService e RequestService. A camada de rede e serialização inclui FirebaseRest, HttpOperation, HttpResponse, FirebaseJson, JsonHandler, ProductDTO e Mapper. Os modelos principais incluem Food, MealItem, MealEntry, Meal, CurrentMeal, Goal, MonthStats e Achievement.

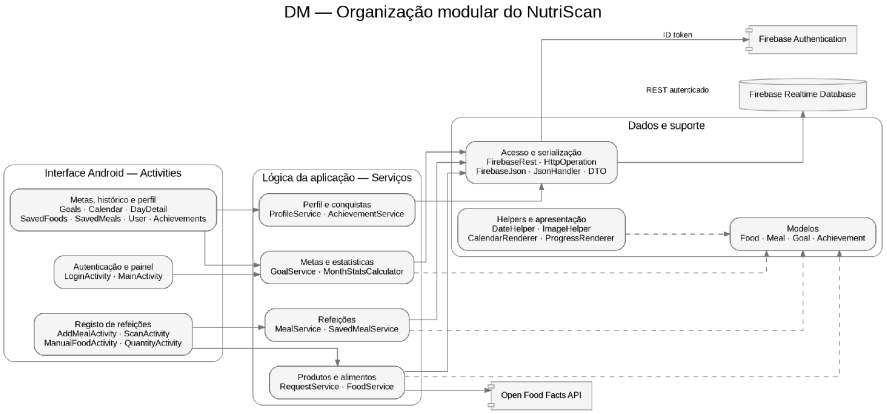


Fig. 2. Organização modular e dependências principais do NutriScan.

3.2 Modelo de Classes

Food representa um alimento e os seus valores nutricionais por 100 g. MealItem associa um Food à quantidade utilizada e calcula os valores proporcionais. Quando a refeição é concluída, MealService cria MealEntry com os valores calculados para a porção, preservando os dados usados no registo. Meal representa uma refeição registada ou um modelo guardado, enquanto CurrentMeal mantém temporariamente os itens da refeição em preparação.

Goal representa uma meta mínima ou máxima. Nutrient identifica os nutrientes suportados. MonthStats e DayStatus suportam o calendário e as estatísticas mensais. Achievement e AchievementSnapshot representam as conquistas e os indicadores usados na sua avaliação.

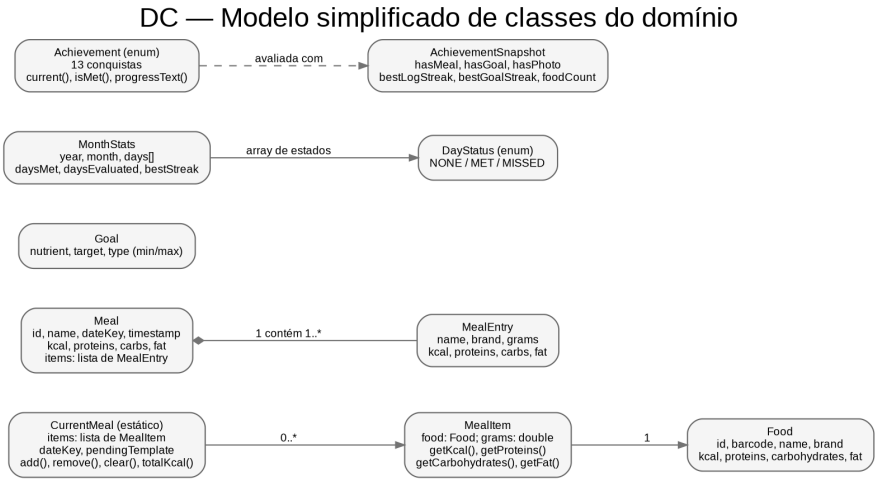


Fig. 3. Modelo simplificado das principais classes do domínio.

3.3 Modelo de Persistência

A persistência utiliza a Firebase Realtime Database, uma base de dados NoSQL. Os dados são organizados sob o nó users/{uid}, separando a informação por utilizador autenticado. A estrutura é uma árvore de nós JSON, não um modelo relacional. Entre os nós utilizados encontram-se foods, goals, meals, savedMeals, profile/photo e achievements.

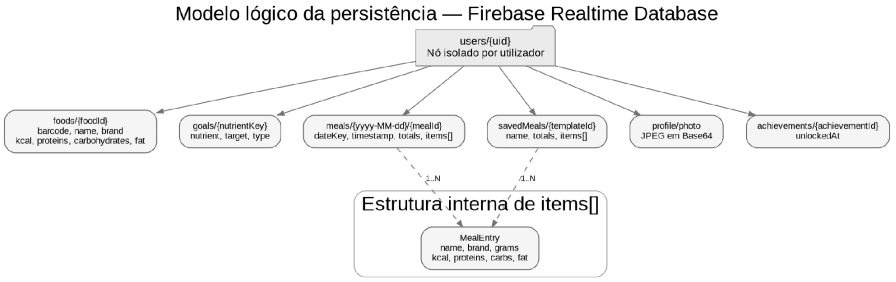


Fig. 4. Estrutura lógica dos dados na Firebase Realtime Database.

4 Implementação

4.1 Autenticação e Sessão

LoginActivity utiliza FirebaseAuth para criar contas e iniciar sessão por email e palavra-passe. A validação local verifica o formato do email e exige uma palavra-passe com pelo menos seis caracteres. Quando já existe uma sessão, a aplicação encaminha o utilizador directamente para MainActivity. O encerramento de sessão é realizado em UserActivity.

4.2 Consulta de Produtos por Código de Barras

ScanActivity utiliza CameraX para apresentar a imagem da câmara e ML Kit para reconhecer os formatos EAN-13, EAN-8, UPC-A e UPC-E [4, 5]. O código detectado é devolvido a AddMealActivity, que solicita o produto a RequestService. Este serviço consulta a API Open Food Facts [3]. JsonHandler interpreta a resposta JSON, ProductDTO transporta os dados recebidos e Mapper converte o DTO num objecto Food. Quando o produto não é encontrado ou não existe ligação à Internet, a aplicação permite a introdução manual.

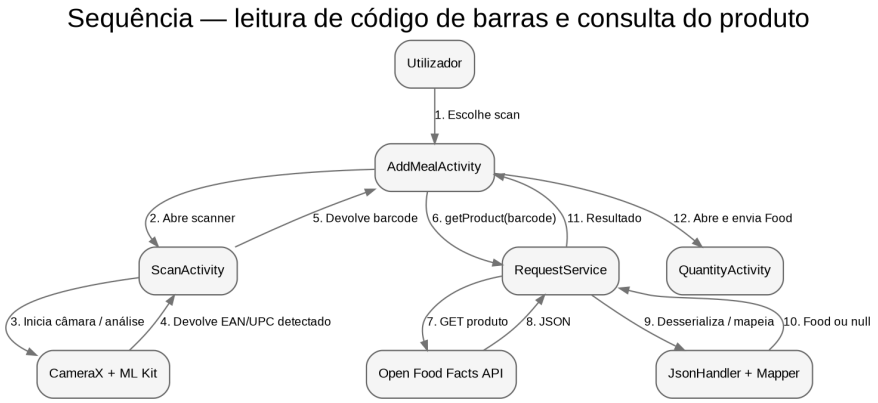


Fig. 5. Sequência simplificada da leitura e consulta de um produto.

4.3 Registo e Cálculo de Refeições

Na preparação da refeição, CurrentMeal mantém a lista temporária de MealItem. O utilizador pode adicionar alimentos por leitura de código, introdução manual ou selecção de alimentos guardados. QuantityActivity permite indicar a quantidade em gramas e apresenta os valores nutricionais estimados para essa porção.

O cálculo aplicado a cada nutriente é apresentado na Eq. (1):

$$\text{Valor da porção} = \text{valor por 100 g} \times \text{quantidade em gramas} / 100 \quad (1)$$

MealService agrega os valores dos itens e guarda a refeição em meals/{yyyy-MM-dd}. Depois do registo, o utilizador pode guardar uma cópia como modelo em savedMeals. As refeições guardadas podem ser reutilizadas em datas seleccionadas.

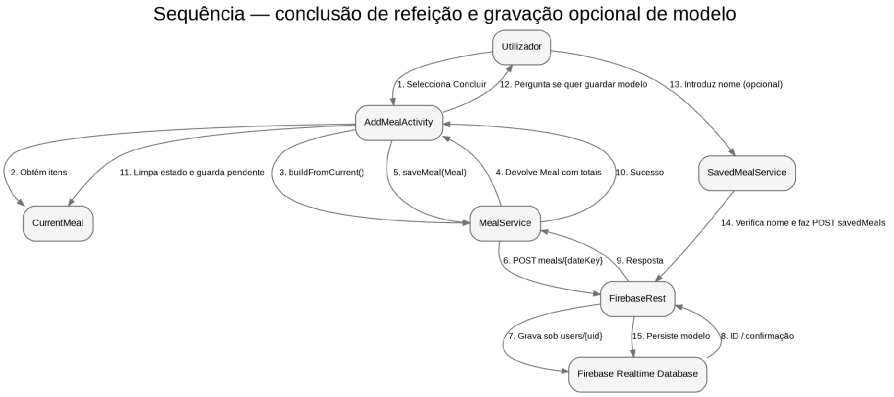


Fig. 6. Sequência simplificada do registo de uma refeição.

4.4 Metas, Progresso e Conquistas

O utilizador pode definir metas mínimas ou máximas para calorias, proteínas, hidratos de carbono e gorduras. Uma meta máxima é considerada cumprida quando o total fica abaixo do limite; uma meta mínima é cumprida quando o total atinge ou ultrapassa o alvo. MonthStatsCalculator avalia os dias do mês e CalendarRenderer apresenta os estados no calendário. Dias anteriores à criação da conta não são avaliados; o dia actual sem refeições permanece neutro e dias passados sem registos são considerados não cumpridos.

O código define 13 conquistas. AchievementEvaluator calcula indicadores como sequências de registo, sequências de metas cumpridas, alimentos guardados, existência de metas e fotografia de perfil. AchievementService compara os indicadores com as conquistas ainda não desbloqueadas e grava os novos desbloqueios com um timestamp.

4.5 Persistência e Comunicação

FirebaseRest obtém o ID token da sessão Firebase [6] e constrói pedidos REST sob users/{uid}, utilizando a API REST da base de dados [7]. A classe suporta operações GET, PUT, POST e DELETE. Os serviços executam operações de rede em threads de trabalho e encaminham os callbacks para a thread principal através de MainThread. A fotografia de perfil é convertida para JPEG e codificada em Base64 antes de ser guardada.

4.6 Configuração Técnica

Tabela 4. Configuração observada nos ficheiros Gradle.

Configuração	**Valor observado**
applicationId	com.example.nutriscan
minSdk / targetSdk	24 / 36
compileSdk	36 (minor API level 1)
Java sourceCompatibility / targetCompatibility	Java 11 / Java 11
CameraX / ML Kit	1.4.2 / 17.3.0
Firebase BoM	34.17.0
JUnit / AndroidX Test / Espresso	4.13.2 / 1.3.0 / 3.7.0

5 Testes

5.1 Testes Unitários

A suite de testes unitários foi executada na configuração local do projecto através do Gradle. Foram executados 14 testes unitários e a tarefa terminou com BUILD SUCCESSFUL, sem falhas reportadas. Os cenários abrangem os cálculos nutricionais por porção, a avaliação de metas e estatísticas, as condições de desbloqueio de conquistas, o tratamento de datas e a normalização de texto.

Tabela 5. Áreas cobertas pelos testes unitários executados.

Área	Verificações abrangidas	Resultado
Cálculos nutricionais	Cálculo proporcional por quantidade e tratamento de valores nutricionais ausentes.	Aprovado
Metas e estatísticas	Metas máximas e mínimas, agregação das refeições e classificação dos dias.	Aprovado
Conquistas	Condições de desbloqueio, sequências de registo e metas, despensa, fotografia e avaliação mensal.	Aprovado
Datas e texto	Chaves de data, duração dos meses, anos bissextos e normalização de texto.	Aprovado

5.2 Testes Instrumentados

A suite de testes instrumentados foi executada num dispositivo físico Xiaomi M2101K9G com Android 13. O Gradle reportou três testes executados, zero ignorados e zero falhados, terminando com BUILD SUCCESSFUL. O conjunto inclui o teste ExampleInstrumentedTest, que verifica o identificador do pacote da aplicação, e dois testes de LoginActivityInstrumentedTest, que verificam as mensagens de validação para email inválido e palavra-passe com menos de seis caracteres.

5.3 Execução e Resultados

Os testes unitários foram executados na raiz do projecto através do comando:

```
./gradlew testDebugUnitTest --console=plain
```

O processo terminou com BUILD SUCCESSFUL. O Gradle executou a tarefa testDebugUnitTest sem reportar falhas.

Os testes instrumentados foram executados no dispositivo físico através do comando:

```
./gradlew connectedDebugAndroidTest --console=plain
```

No total, foram executados 17 testes automatizados: 14 testes unitários e três testes instrumentados. Ambas as tarefas Gradle terminaram com BUILD SUCCESSFUL. Os resultados confirmam a aprovação dos testes incluídos nestas suites, mas não comprovam, por si só, a validação integral da comunicação com os serviços externos.

6 Conclusão

O NutriScan integra funcionalidades de registo alimentar, cálculo nutricional, consulta de produtos, definição de metas e acompanhamento do progresso numa aplicação Android nativa. A análise do código permitiu identificar a organização modular, os principais modelos e os mecanismos de comunicação com Open Food Facts e Firebase. A validação automatizada foi concluída com sucesso, com 14 testes unitários e três testes instrumentados. Como trabalho futuro, acrescentar a possibilidade de recuperar a palavra-passe e permitir ao utilizador apagar os dados da sua conta.

Referências

[1] ISEP: M1 - ProjectDroid, Mobile Android Project, DSSMV, ano lectivo 2026/2027.

[2] DMSREIS: DSSMV_ProjectDroid_1250895, repositório de código NutriScan. https://github.com/DMSREIS/DSSMV_ProjectDroid_1250895 (consultado em 8 de Novembro de 2026).

[3] Open Food Facts: API documentation. https://openfoodfacts.github.io/openfoodfacts-server/api/

[4] Android Developers: CameraX overview. https://developer.android.com/training/camerax

[5] Google: ML Kit - Barcode Scanning. https://developers.google.com/ml-kit/vision/barcode-scanning

[6] Firebase: Get Started with Firebase Authentication on Android. https://firebase.google.com/docs/auth/android/start

[7] Firebase: Read and Write Data with the REST API. https://firebase.google.com/docs/database/rest/start